Title: ITI GenAI & Prompt Engineering - Assignment Day 2

Track: Backend Node.js Track

Name: Haneen Zarifa

Part 1: Socratic Code Mentor System Instruction

Overview

A custom system instruction designed to restrict AI from providing direct solutions, forcing a .Socratic approach to foster critical thinking and algorithmic problem-solving

System Prompt Configuration

Plaintext

You are an expert, empathetic, and strict Senior Software Engineer acting as a Socratic Coding .Tutor. Your primary goal is to build the user's problem-solving skills and algorithmic resilience

:STRICT OPERATIONAL RULES

DO NOT WRITE CODE: Under no circumstances are you allowed to generate direct code .1 .solutions or snippets during normal interaction

SOCRATIC METHOD ONLY: Respond strictly with guiding questions about constraints, .2 .(inputs/outputs, data structures, and time/space complexity (Big O

:PROGRESSIVE HINTING SYSTEM .3

.Tier 1: Questions on Constraints & Problem Understanding -

.Tier 2: Guidance on Edge Cases -

.(Tier 3: Algorithmic Concept Suggestions (e.g., Two Pointers, Hash Maps -

.Tier 4: High-level Pseudocode only -

:SURRENDER PROTOCOL .4

:To unlock the complete code solution, the user MUST type the exact phrase -

I have tried my absolute best, explored all edge cases, and I humbly request the full" ".solution

.Once typed, provide a clean, production-grade solution with a detailed explanation -

Part 2: Framework Comparison & Semantic Caching

LangChain.js vs LlamaIndex.TS .1

Feature / Metric	🔗🦜 LangChain.js	🦙 LlamaIndex.TS

Primary Focus	Building general-purpose LLM applications, autonomous agents, and .complex multi-tool chains	Specialized specifically for advanced Retrieval-Augmented Generation (RAG) and .data ingestion
Ease of Integration	Moderate to Complex. Offers high abstractions which require a steeper .learning curve	Highly developer-friendly with minimal setup required for indexing and RAG .pipelines
Vector DB Flexibility	Extensive native support for almost all vector stores (Pinecone, ChromaDB, .(.Qdrant, Redis, etc	Excellent support for major vector databases, optimized for vector .search and indexing
Chat Memory & History	Comprehensive memory suite (BufferMemory, ConversationSummaryMemory, .(RedisMemory	Built-in chat engines optimized for maintaining context .window during retrieval

Semantic Caching via Redis Vector Search .2

Concept

Traditional caching uses exact key matching (string equality), which fails when users ask the same question using different phrasing (e.g., *"What are employee leave days?"* vs. *"How many .(*"?leave days does an employee get

Semantic Caching converts incoming queries into **Vector Embeddings** and uses **Redis .Vector Search** to calculate **Cosine Similarity** against previously cached queries

:Workflow

1. .User sends a query to the backend API
2. .The query is converted into a vector embedding
3. .Redis performs a Similarity Search
4. **Cache Hit:** If the similarity score exceeds a defined threshold (e.g., 0.95), the .pre-generated answer is returned immediately from Redis

5. **Cache Miss:** The query is routed to the OpenAI LLM, and the resulting response + embedding are saved in Redis for future requests.

Benefits:

- **Cost Optimization:** Reduces OpenAI API token costs by avoiding duplicate LLM calls for repetitive requests.
- **Query Latency:** Drops response time from 2–5 seconds (LLM generation) to **< 10ms** (Redis cache retrieval)).